Transportation Research Part E 172 (2023) 103080

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

Optimization of ride-sharing with passenger transfer via deep reinforcement learning

Dujuan Wang [a], Qi Wang [a], Yunqiang Yin [b,*], T.C.E. Cheng [c]

[a] Business School, Sichuan University, Chengdu 610064, China
[b] School of Economics and Management, University of Electronic Science and Technology of China, Chengdu 610064, China
[c] Department of Logistics and Maritime Studies, The Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong

ARTICLE INFO

Keywords:
Ride-sharing
Passenger transfer
Deep reinforcement learning
Fleet management

ABSTRACT

With the emergence of the sharing economy and the rapid growth of mobile communications technologies, many novel sharing service models have been developed stemming from ride-hailing. Urban traffic congestion, coupled with energy conservation and emissions reduction, has prompted research on enhancing vehicle seat utilization in taxi service. To offer more effective and reliable ride-hailing, we consider ride-sharing problem with passenger transfer that allows passegers to transfer between vehicles at transfer stations. The problem requires simultaneous addressing the issues of request dispatching, transfer scheduling, and vehicle rebalancing. Studying such a ride-hailing model, we propose a novel joint decision framework combining deep reinforcement learning (DRL) with integer-linear programming (ILP) to solve the problem. We use ILP to obtain the optimal online dispatching and matching strategy in each decision stage, and DRL to learn the approximate state value of each vehicle that incorporates with some strategies to limit the state space and reduce the computational complexity. Performing numerical studies on the real-world trip dataset in Chengdu, we demonstrate that the proposed method outperforms several state-of-the-art methods, and that ride-sharing with passenger transfer is more beneficial than traditional ride-sharing.

1. Introduction

Recently, with advances in the mobile technologies, we witness the growth of ride-hailing platforms, such as DiDi, Uber, Lyft, and so on. The size of the ride-hailing market is growing very fast: the ride-hailing order quantity in China exceeded 0.76 billion in only one month in 2021 (Ministry of Transport of the People's Republic of China, 2021). Ride-sharing has enormous potential to benefit all the participants and society. For example, it is reported that Uber Pool can save one million miles a day with ride-sharing (Uber, 2022). Considering a share ability network in New York, Santi et al. (2014) showed that the cumulative trip length can be reduced 40% with just a relatively small number of passengers being unsatisfied and that the system can still be effective with fewer taxis. Alisoltani et al. (2021) studied the ride-sharing effect on road congestion, and showed that the ride-sharing system can help significantly alleviate traffic congestion based on a large-scale simulation in Lyon. Ke et al. (2020) pointed out that the ride-sharing market has a larger service rate than the traditional no-pooling market, and showed ride-sharing can reduce the time cost of both ride-sharing passengers and private car drivers in the transport system under some scenarios. Ke et al. (2021) showed that the ratio of vehicle routing distance

* Corresponding author.
E-mail address: yinyq@uestc.edu.cn (Y. Yin).

https://doi.org/10.1016/j.tre.2023.103080
Received 17 September 2022; Received in revised form 5 February 2023; Accepted 18 February 2023
Available online 28 February 2023
1366-5545/© 2023 Elsevier Ltd. All rights reserved.

in ride-sharing to non-shared trip distance is inversely proportional to a logarithm form of passenger demand. Lokhandwala and Cai (2018) considered shared autonomous taxis in New York City, and showed that the model can reduce both the fleet size and accumulative travel distance while maintaining the service level.

However, due to the dispersion of service requests, the number of requests with similar routes is usually small, which results in a low sharing rate in the ride-sharing system. To deal with the problem, we propose a novel sharing model based on the ride-sharing system that allows passenger transfer between vehicles. The idea has been adopted by Uber in crowded cities such as Seattle (Hawkins, 2015), and widely used in other areas such as logistics service (Cortés et al., 2010; Azcuy et al., 2021; Manchella et al., 2021). The new sharing model has more potential to improve the sharing-rate (utilization of fleets) and service level, which reduces the average waiting time of passengers by about 40% and increases the utilization rate of vehicles by about 20% compared with the classical ride-sharing model (Singh et al., 2022), and to increase ride availability and city connectedness (Teubner and Flath, 2015).

In this paper we consider the problem of ride-sharing with passenger transfer over a finite time horizon. The decisions consist of three kinds of scheduling tasks: (i) *request dispatching*, i.e., matching vehicles with requests in real time; (ii) *transfer scheduling*, i.e., determining which requests need to be transferred and which transfer stations are these transferred requests sent to; and (iii) *vehicle rebalancing*, i.e., repositioning some idle vehicles to high demand areas for rebalancing supply with demand, subject to the maximum passenger waiting time, maximum delay time, maximum vehicle capacity, and maximum transfer station capacity constraint constraints. The objective is to minimize the waiting times of passengers, delay times of passengers, transfer times of accepted requests, and dead times of vehicles, and maximize the incentives for serving requests. Since the information of each request is revealed in real time, we model the problem as a sequential decision-making problem, where each individual decision of matching a vehicle to a request is based on two factors, namely an instant reward for the vehicle serving the request attained from real-time information, and a factor denoting the impact of this decision on the future. To solve this problem, we use deep reinforcement learning (DRL) to determine the state value of each vehicle to overcome the difficulty of "dimensionality curse", and develop a centralized dispatch optimization framework by solving an integer-linear programming (ILP) model to make the decision in each stage. Our main contributions are as follows:

1) We model the ride-sharing problem with passenger transfer as a multi-stage sequential decision-making problem, simultaneously addressing the issues of request dispatching, transfer scheduling, and vehicle rebalancing to determine: (i) which vehicles serve each request, (ii) the trip of each vehicle after picking up the passengers in a request, (iii) the transport mode, i.e., without transfer mode or with transfer mode, for each accepted request, and (iv) the pickup times and the final completion time of each accepted request.
2) We develop a DRL model to solve the problem in a stable and efficient manner, where an ILP model is developed to find the optimal online dispatching and matching strategy in each decision stage, and a neural network is used to learn the approximate state value of each vehicle, a trip search strategy is introduced for each vehicle after picking up the passengers in a request, the experience replay mechanism is exploited to enhance the stability of training, and the ε-greedy policy is used to guarantee convergence of the algorithm.
3) We conduct extensive numerical studies using the real-world trip dataset in Chengdu, demonstrating that ride-sharing model with passenger transfer is more beneficial than the traditional ride-sharing model, and that the proposed method outperforms the model-free based DRL algorithm in Singh et al. (2022) for solving a similar ride-sharing problem with passenger transfer.

We organize the rest of the paper as follows: In Section 2 we review the literature related to our study. In Section 3 we introduce the problem and formulate the mathematical model. In Section 4 we discuss the proposed dispatch framework for the problem and policy evaluation with reinforcement learning in detail. In Section 5 we discuss the results of extensive numerical studies based on the real-word trip dataset in Chengdu. Finally, in Section 6, we conclude the paper and suggest topics for further research.

2. Literature review

Ride-hailing and ride-sharing systems have received much research attention for many years, the main characteristic of which is that multiple passengers with different origins or destinations will share a vehicle for a portion of the ride. For a comprehensive review of recent studies on various ride-hailing/sharing systems, we refer the reader to Agatz et al. (2012), Wang and Yang (2019), Narayanan et al. (2020), and Chen et al. (2020). In addition to the ride matching problem, other problems about dispatching are also a hot point in shared vehicles service, such as idle vehicle reposition (Braverman et al., 2019) and the new model that incorporates the passenger's work (Fielbaum et al., 2021; Yao et al., 2021; Fielbaum, 2021). Recently, there are some works focusing on the problem about how such platforms affect the efficiency of the transport versus the traditional street-hailing method (Feng et al., 2021).

For the ride matching problem, the rolling horizon strategy is generally used to dispatch vehicles (Alonso-Mora et al., 2017; Simonetto et al., 2019), and the optimal or heuristic dispatch policies are developed in each rolling. Some works adopt the re-optimization strategy in each iteration to maximize the optimization objective (Li et al., 2021). Alonso-Mora et al. (2017) constructed an available matching graph for vehicles and requests, formulating ride-sharing as an assigning problem, and applying ILP to solve the matching problem. Bertsimas et al. (2019) employed the rolling horizon strategy for the online vehicle routing problem and developed an optimization framework, which has been proven to be effective in large-scale applications. However, the developed algorithms in the above studies are all myopic, which lacks the ability to capture the uncertain information in the future and does not consider the impact of current assignment on future assignments.

There are studies that apply approximate dynamic programming (ADP) to solve the ride-hailing and ride-sharing problems. Al-Kanj

et al. (2020) studied ride-hailing with autonomous electric vehicles, and developed an ADP algorithm to tackle the dispatching problem, simultaneously addressing the issues of request assignment, price surge, and fleet management. Yu and Shen (2020) formulated the ride-pooling problem as a multi-stage stochastic program where a vehicle can be shared by at most two passenger groups, and used ADP to solve the assignment problem between passengers and vehicles, where a decomposition scheme is developed to accelerate the proposed algorithm.

Recently, reinforcement learning (RL), which has been successfully applied for solving various computationally challenging optimization problems, has been shown to be a powerful tool for solving the ride-hailing and ride-sharing problems, where RL is applied to learn the spatial and temporal features of demands (Cao et al., 2020; Asadi and Pinkley, 2021). Lin et al. (2018) developed a contextual multi-agent reinforcement learning framework consisting of contextual deep Q-learning and contextual multi-agent actor-critic to achieve explicit coordination among plentiful vehicles on a large-scale on-demand ride-hailing platform. They showed the effectiveness of the proposed method over state-of-the-art methods based on the real-word dataset from DiDi ChuXing. Tang et al. (2019) developed a cerebellar value network for multi-driver order dispatching in the ride-sharing system to enhance the stability of state value iteration with nonlinear function approximators. They demonstrated that the developed method attains dramatically improvement on both total driver income and passenger experience-related metrics. Holler et al. (2019) developed a DRL model to solve the dispatching and reposition problems from the system-centric perspective, where a central fleet management agent is responsible for decision-making for all the vehicles. Jiao et al. (2021) also proposed a DRL model for real-world large-scale fleet management. Shi et al. (2020) investigated on-demand ride-hailing service with electric vehicles, developed an RL model to estimate the value function, and an ILP model to derive the vehicle fleet coordination policy in each decision epoch. Liang et al. (2021) applied DRL combined with ILP to find the optimal schedule, involving electric vehicle charging, order dispatching, and vehicle rebalancing for a large-scale shared electric vehicle fleet operator, where a constrained rebalancing strategy is developed to enhance the exploration efficiency of training. Feng et al. (2022) Combining ride-sourcing and public transit services, and proposed a reinforcement learning approach. Shah et al. (2020) proposed a neural ADP framework for the ride-pooling system, where ADP in conjunction with a neural network is used to learn the approximated value functions, and deep Q-network (DQN) is used to stabilize the neural ADP. The above studies are all based on a central dispatch model, while recently Al-Abbasi et al. (2019) developed a model-free approach based on DQN for the ride-sharing system. In this framework, each vehicle makes its own decision based on the impact of the decision using Q-leaning without coordination with the other vehicles. Based on this framework, Haliem et al. (2021) integrated the price problem into the ride-sharing system.

Incorporating passenger transfer into ride-sharing significantly increases modelling and computational complexity, but it can efficiently improve the service efficiency of vehicles and reduce the cost of passengers. Recently, there are some studies on ride-sharing with passenger transfer. Teubner and Flath (2015) discussed the economics of ride-sharing with passenger transfer in detail, including competition, pricing, and platform envelopment, and showed that ride-sharing with passenger transfer is competitive versus other transport models. Masoud and Jayakrishnan (2017) formulated the ride-sharing with passenger transfer problem as a binary program, where pre-processing is used to simplify the network, and developed a decomposition algorithm to solve the ride-matching problem. Singh et al. (2022) developed a distributed model-free algorithm for the ride-sharing with passenger transfer problem, where DRL is applied to learn the optimal matching policies. They showed that significant cost saving is achieved by allowing passengers to transfer between vehicles.

To the best of knowledge, Singh et al. (2022) is most related to our study. However, compared with our study, from the model perspective, Singh et al. (2022) aimed to find the optimal dispatching policy that dispatches vehicles to regions where future demand is anticipated, and matching policy to minimize the gap between supply and demand, total dispatch time, extra travel times of passengers, number of transfer stations for passengers, and number of vehicles in the fleet that change their status from being inactive to active, subject to the maximum vehicle capacity constraint. In contrast, our study seeks to minimize the waiting times of passengers, delay times of passengers, transfer times of accepted requests, and dead times of vehicles, and maximize the incentives for serving requests, subject to the maximum passenger waiting time, maximum delay time, maximum vehicle capacity, and maximum transfer station capacity constraint constraints. From the solution method perspective, Singh et al. (2022) developed a distributed model-free algorithm that uses DRL to learn the optimal policies for each vehicle individually by interacting with the external environment, where a greedy strategy is adopted to match passengers and vehicles without considering the impact of the current matching on future matchings. Our study models the problem as a Markov decision process, and develops a framework that combines DRL with ILP to find near-optimal solutions, where neural network is applied to evaluate the state values of vehicles, and the online dispatching and matching problem is optimally solved by formulating the problem as an ILP model. The results of numerical studies based on the real-word dataset from DiDi Chuxing show that our solution method outperforms that developed in Singh et al. (2022).

3. Problem description and formulation

3.1. Problem description

The ride-sharing with passenger transfer problem is characterized by the need to dynamically dispatch a fleet of vehicles $\mathscr{V} = \{1, \ldots, N\}$ with random initial locations to meet service requests $\mathscr{R}$ arriving randomly over a finite planning horizon T and within a service region. Each location in a location set $\mathscr{G} = \{1, 2, \ldots, G\}$, can serve as a trip origin or destination, and the locations in a subset $\mathscr{T}$ of $\mathscr{G}$ can serve as transfer stations. The trip information of each request, including the number of passengers, and the origin locations and destinations of passengers, is sent to a central system in real time, which collects the request information during a time window or decision epoch Δt. More than one group of passengers can carpool in a vehicle at the same time, and passengers can finish their trips

through multiple transfers. Considering that multiple transfers in one request will reduce customer satisfaction, we restrict that each request can be transferred at most once. It means that a group of passengers associated with a request may be first picked up by a vehicle at the origin location, and transported directly to the destination, or transported to a transfer station and the vehicle or another vehicle continues to serve the remaining trip.

The constraints considered in our study include (1) *passenger waiting time constraint*: the waiting time for each request before being picked up by a vehicle cannot exceed a time limit Δ_w; (2) *delay time constraint*: the delay time for each request, i.e., the difference between the actual arrival time at its destination and the earliest arrival time at the destination via the shortest path between its origin location and destination, which can be calculated by a shortest path algorithm without any waiting time, cannot exceed a predefined value Δ_d; (3) *vehicle capacity constraint*: each vehicle has a maximum passenger capacity $\mathscr{N}^{max}$, implying that the number of passengers in each vehicle cannot exceed $\mathscr{N}^{max}$ at any time; and (4) *transfer station capacity constraint*: each transfer station has a maximum transfer service capacity T^{max} due to the physical constraints of transfer stations, implying that there are at most T^{max} transfer requests that can arrive at each transfer station in a time stage. Once a request is accepted by the system, it must be completed under the passenger waiting time and delay time constraints.

The goal is to make sequential decisions to dispatch the vehicles to serve the dynamically arriving trip requests over a finite planning horizon T, so as to minimize the waiting times of passengers, delay times of passengers, and dead times of vehicles, and maximize the incentives for serving requests, subject to the aforementioned constraints.

Fig. 1 shows an example of the ride-sharing with passenger transfer problem. In this example, there are two vehicles $v1$ and $v2$, three requests with origin nodes $o1$, $o2$ and $o3$, and destinations $d1$, $d2$ and $d3$, and a transfer station s. Due to the maximum waiting time constraint, vehicle $v1$ cannot serve request 3, whereas vehicle $v2$ cannot serve requests 1 and 2, each of which has one passenger. We show the ride-sharing strategies and service route of each vehicle under two scenarios, i.e., not allowing transfer and allowing transfer at transfer station s in Fig. 1(a) and 1(b), respectively. In Fig. 1(a), due to the maximum delay time constraint, vehicle $v1$ cannot simultaneously serve passengers 1 and 2, so passenger 1 or 2 is left to be served by another vehicle or unserved. In contrast, in Fig. 1(b), all the passengers can be served by the two vehicles under the transfer strategy. Specifically, vehicle $v1$ picks up passengers 2 and 1 at nodes $o2$ and $o1$ one by one, respectively, drops off passenger 1 at transfer station s, and then takes passenger 2 to the destination. Vehicle $v2$ picks up passengers 3 and 1 at node $o3$ and transfer station s one by one, and then takes them to destinations $d3$ and $d1$, respectively.

3.2. Problem formulation

In this section we model the problem as a Markov decision process (MDP), where each vehicle is regarded as an agent and the tasks of each agent are divided into two parts, i.e., request related and fleet management related, and each agent interacts with the environment in each time stage $t = 1, 2, \ldots T$ with duration $\Delta\delta$. For each vehicle, there are two kinds of tasks: the request-related tasks and fleet management-related tasks. Specifically, the request-related tasks include accepting new request and transporting the corresponding passengers to it's destination or a transfer station, and continuing to transport the passengers in the vehicle to their destinations. The fleet management-related tasks include repositioning the vehicle if it is idle to other regions and waiting in the same place.

We define the state space, action domain, state transition function, and reward function of the MDP for the problem under consideration.

(1) *State Space*: We use a tuple of stage vectors $S_t = (\mathscr{V}_t, \mathscr{R}_t)$ to present the state of the system in time stage t, where $\mathscr{V}_t = (v_1, \ldots, v_N)$ denotes (resp., $\mathscr{R}_t = (r_1, \ldots, r_M)$) the tuple of state vectors of all the vehicles (resp., all requests) in time stage t. Here, M is the number of new arrival requests waiting to be served in time stage t and v_i (resp., r_j) contains the information on the ith vehicle (resp., jth waiting request). The requests that have not been accepted and cannot be served by any vehicle in a decision epoch due to the waiting time and delay time constraints will be removed from the system. Each state vector v_i in $\mathscr{V}_t$ corresponding to the ith vehicle is represented by an attribute vector with three tuples $v_i = (\ell_i, \mathscr{L}_i, \mathscr{T}_i, \mathscr{U}_i)$, where ℓ_i gives the current location of the vehicle, $\mathscr{L}_i$ gives a location list (pick-up location or drop-off location) that the vehicle will visit, and $\mathscr{T}_i$ denotes a time list that gives the latest arrival time to corresponding locations to pick-up or drop-off corresponding request, $\mathscr{U}_i$ denotes the remaining available capacity in vehicle when leaving the corresponding locations. For example, if the ith vehicle is going to serve a new request r_j, then $\ell_i = (o_j, d_j)$; and if the vehicle is stationary, then $\ell_i = \varnothing$. Each state vector r_j in $\mathscr{R}_t$ corresponding to a waiting request is represented by an attribute vector with six

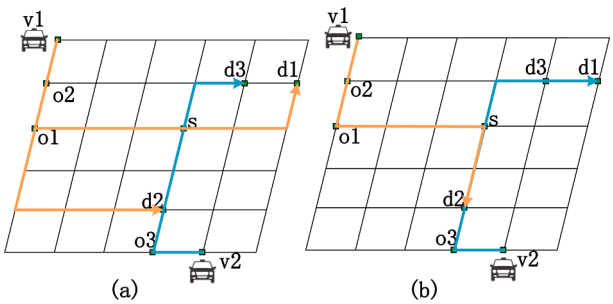

Fig. 1. Schematics of two hypothetical ride-sharing scenarios: (a) without transfer, (b) with transfer.

tuples $r_j = (o_j, d_j, N_j, \tau_j, R_j, t_j)$, where o_j denotes the original location, d_j denotes the destination, N_j is the number of passengers in the request, τ_j is the transfer times of the request, R_j measures the value of request r_j, t_j measures the time at which the request arrives at the system, i.e., the request time.

(2) *Action Domain:* Let $\mathscr{A}^f_{v,t}$ denote the set of all the feasible actions of vehicle v in time stage t. Here, a feasible action means that the resulting state vector must satisfy the maximum wait time, maximum delay time, and vehicle capacity constraints. A feasible action of system in time stage t is denoted by a set $\mathscr{A}_t = (a_{v,t})_{v \in \mathscr{V}_t}$, including the action vector of all the vehicles in $\mathscr{V}_t$ involved in time stage t, where $a_{v,t} \in \mathscr{A}^f_{v,t}$ denotes the specific action taken by vehicle v. A vehicle $v \in \mathscr{V}_t$ can take one of the following actions:

- *Pass*: An empty instruction, which means that the *Pass* action does not work on the vehicle. Under this action, the vehicle will stay in the current location that it is located if $|\mathscr{L}_v| = 0$, and continue to travel according to the original schedule otherwise. In other words, the state of the vehicle evolves naturally without any other external influence.

$Assign_{v,\,t}\,(r, trip)$: This action assigns a request $r \in A^o_{v,\,t}$ to vehicle v that travels in $trip \in Trip_{v,\,r,\,t}$, where $A^o_{v,\,t}$ is the set of requests in time stage t that can be assigned to vehicle v directly to transport them to their destinations with respect to the waiting time, delay time, and vehicle capacity constraints, and $Trip_{v,\,r,\,t}$ is the set of feasible trips of vehicle v after picking up the passengers in request r that is generated by the dynamic trip generation method developed in Alonso-Mora et al. (2017). Note that given $trip \in Trip_{v,\,r,\,t}$, we can determine whether the passengers in request r are transferred and at which transfer station to transfer if transferred, and that there is a set of feasible assignment actions to choose from, each of which corresponds to a combination $(r, trip)$ with $r \in A^o_{v,\,t}$ and $trip \in Trip_{v,\,r,\,t}$.

- *Reposition* o': This action repositions vehicle v to a nearby zone $o' \in A^z_{v,\,t}$ if the vehicle is idle more than a certain period of time, where $A^z_{v,\,t}$ is the set of nearby zones that can be assigned to vehicle v in time stage t, and z is the current location of the vehicle. Note that some imbalance may occur between the new requests and available vehicles in certain zones due to the waiting time, delay time, and capacity constraints, which results in some requests in a zone being unserved by the available vehicles in the zone. In this case, this action will dispatch some idle vehicles from the nearby zones to the zone. For example, as shown in Wang and Noland (2021), the number of the pick-up requests is much greater than that of the drop-off requests within the 3rd ring in Chengdu, China, during the evening peak. This means that there exists a gap between the requests and available vehicles in this region, and that the idle vehicles in the nearby zones need to be repositioned to this region to increase the service capacity of this region, so an advanced *Reposition* is needed.

To ease the presentation, let $x_{v,\,a}(S)$ be a binary variable equal to 1 if and only if vehicle v takes action a in state S. We can characterize the whole decision on state S in time stage t by a set of binary variables as

$$\mathscr{X}_t(S) = \left\{ x_{v,a}(S) \in \{0,1\}^{\sum_{v \in \mathscr{V}_t} |\mathscr{A}^f_{v,t}|} : \sum_{a \in \mathscr{A}^f_{v,t}} x_{v,a}(S) = 1 \forall v \in \mathscr{V}_t \right\}$$

which states that each vehicle must take exactly one action.

(3) *State Transition*: Let $\mathscr{R}^{new}_t$ be the set of new requests collected between time stage t and stage $t+1$, including the new requests entering into the system and the requests assigned to a transfer station that will arrive at the station in time stage $t+1$. As a result, we express the complete dynamic process of this system as follows:

$$\left\{ \left(S_t, \mathscr{A}_t, \mathscr{R}^{new}_{t+1}, S_{t+1} \right) : t = 1, 2, \ldots, T-1 \right\}$$

The transition from state S_t to pre-decision state S_{t+1} depends on the action vector $\mathscr{A}_t$ and the exogenous information $\mathscr{R}^{new}_{t+1}$. Thus, we write $S_{t+1} = T\left(S_t, \mathscr{A}_t, \mathscr{R}^{new}_{t+1}\right)$, where $T(.,.,.)$ is the transition function showing how the system state evolves over time. Once the action of each vehicle is determined, the state of the vehicle will be transited by the simulator.

(4) *Reward Function*: The objective function is composed of five components: (i) the total incentive for serving the accepted requests $\sum_{r \in R_{accept}} R_r$, where R_{accept} is the set of accepted requests that will be served by one or two vehicles; (ii) the total waiting time of all the accepted requests $\sum_{r \in R_{accept}} T_{wait-r}$, where T_{wait-r} is the total waiting time of request r, where the waiting time is measured by the difference between the pickup time and request time; (iii) the total delay time of all the accepted requests $\sum_{r \in R_{accept}} T_{delay-r}$, where $T_{delay-r}$ is the delay time of request r; (iv) the total penalty cost of transferring $\sum_{r \in R_{accept}} \vartheta n_{\text{transfer}-r}$, where $n_{\text{transfer}-r} \in \{0,1\}$ is the number of transfer times of request r, and ϑ is the penalty cost per request transfer; and (v) the total dead time of the vehicles $\sum_{v \in V} T_{dead-v}$, where T_{dead-v} is the total dead time of vehicle v, which is the sum of the travel times from one location where an idle vehicle v is initially located to a nearby zone. Note that we neglect the dead time of vehicle v to pick up passages within the same zone. The goal is to maximize

$$\sum_{r \in R_{accept}} R_r - w_1 \sum_{r \in R_{accept}} T_{wait-r} - w_2 \sum_{r \in R_{accept}} T_{delay-r} - \sum_{r \in R_{accept}} \vartheta n_{\text{transfer}-r} - w_3 \sum_{v \in V} T_{dead-v} \tag{1}$$

where w_1, w_2, and w_3 are the weight coefficients of the total waiting time of all the accepted requests, the total delay time of all the accepted requests, and the total dead time of the vehicles, respectively.

Let $C_t(S_t, \mathscr{A}_t)$ be the reward obtained from the all the vehicles when taking action $\mathscr{A}_t$ on state S_t in time stage t, and $C_{v,\,t}(S_{v,\,t}, a_{v,\,t})$ the reward obtained from vehicle v when taking action $a_{v,\,t}$ on state $S_{v,\,t}$ in time stage t, where $S_{v,\,t}$ is the state of vehicle v in time stage t. It is evident that $C_t(S_t, \mathscr{A}_t) = \sum_{v \in \mathscr{V}} C_{v,t}(S_{v,t}, a_{v,t})$. The reward function of an individual vehicle is as follows:

The reward for assigning request r to vehicle v that travels in *trip* is

$$C_{v,t}\left(S_{v,t}, a_{v,t} = Assign_{v,t}\ (r, trip)\right) = R_{r,t} - w_1 W_{v,r,t} - w_2 D_{v,r,t} - \vartheta E_{v,r,t} \tag{2}$$

where $R_{r,\,t}$ denotes the incentive for a vehicle serving request r, which is equal to R_r if r is a new arrival request and 0 if r is a request that will arrive at a transfer station in time stage t; $W_{v,\,r,\,t} = \max\{T^p_{v,\,r,\,t} - T^r_{r,\,t}, 0\}$ measures the waiting time incurred in this action, where $T^p_{v,\,r,\,t}$ is the pickup time, and $T^r_{r,\,t}$ is the request time of request r; $D_{r,\,t} = W_{v,\,r,\,t} + X_{v,\,r,\,t} - \|d_r - o_r\|$ denotes the cumulated travel delay up to the end of time stage t, where $X_{v,\,r,\,t}$ is the time of request r served by vehicle v in time stage t, which is equal to $T^d_{v,\,r} - T^p_{v,\,r,\,t}$ if the destination of the request could be reached at time $T^d_{v,\,r} \le t\Delta\delta$ in *trip* and $t\Delta\delta - T^p_{v,\,r,\,t}$ otherwise, $\|d_r - o_r\|$ denotes the minimum travel time between the origin o_r and destination d_r, and $E_{v,\,r,\,t}$ is a binary parameter equal to 1 if and only if request r are transferred in *trip*.

- The reward for the pass action is

$$C_{v,t}\left(S_{v,t}, a_{v,t} = Pass\right) = 0 \tag{3}$$

The reward for repositioning vehicle v to a nearby zone o' is

$$C_{v,t}\left(S_{v,t}, a_{v,t} = Reposition\ o'\right) = -w_3 T_{location-v,o'}, \tag{4}$$

where $T_{location-v,\,o'}$ is the travel time from the current location of vehicle v to zone o' that measures the dead time of vehicle v.

4. Solution method

In this section we introduce the joint decision framework that combines DRL with ILP to solve the MDP formulated in Section 3.

4.1. Designing policies

Let $\pi(.)$ be a function (or policy) that maps the state space of the system to the action space and determines the actions of all the vehicles, where $\pi(S)$ describes which action to take when system is in state S under policy π. The goal is to find the optimal policy to maximize the expected total reward over the entire horizon:

$$\max_{\pi \in \Pi} \left(\sum_{t=0}^{T} C_t(S_t, \pi(S_t))\,|S_0 \right), \tag{5}$$

where Π denotes the set of policies.

To solve the above problem, we develop a hybrid policy combining the *myopic policy* and *value function approximation*. Myopic Policy is a greedy policy, which only considers one time stamp information in the current time stage, while Value Function Approximation focuses on finding a parameterized function that can return the approximation value of an arbitrary state.

1) Myopic policy

This strategy performs in a short-sighted way without considering the impact of the action made in the current decision epoch on the future. For the considered problem, we develop a myopic policy by solving an ILP model that finds the optimal dispatching strategy to maximize the overall reward of all the vehicles when the status of the system is $S_t = (\mathscr{V}_t, \mathscr{R}_t)$. To capture the decisions, we let $x_{v,\,r,\,trip}{}^t$ be a binary variable equal to 1 if and only if request $r \in A^o_{v,\,t}$ to is assigned to vehicle v that travels in $trip \in Trip_{v,\,r,\,t}$. We formulate the ILP model as follows:

$$\max \sum_{r \in \mathscr{R}_t^T} \sum_{v \in \mathscr{V}_t} \sum_{trip \in Trip_{v,r,t}} C_{v,t}\left(S_{v,t}, Assign_{v,t}\ (r, trip)\right) x^t_{v,r,trip} \tag{6}$$

subject to

$$\sum_{r \in \mathscr{R}_t^T} \sum_{trip \in Trip_{v,r,t}} x^t_{v,r,trip} = 1 \forall v \in \mathscr{V}_t, \tag{7}$$

$$\sum_{v \in \mathscr{V}_t} \sum_{trip \in Trip_{v,r,t}} x^t_{v,r,trip} \le 1 \forall r \in \mathscr{R}_t \setminus \mathscr{R}_t^T, \tag{8}$$

$$\sum_{v\in\mathscr{V}_t}\sum_{trip\in Trip_{v,r,t}} x^t_{v,r,trip} = 1\forall r\in\mathscr{R}^T_t, \tag{9}$$

$$\sum_{l=1}^{t}\sum_{r\in\mathscr{R}^T_l}\sum_{v\in\mathscr{V}_l}\sum_{trip\in Trip_{v,r,l}} a^l_{v,r,trip}x^l_{v,r,trip} \le T^{max}\forall k\in\mathscr{T} \tag{10}$$

$$x^t_{v,r,trip}\in\{0,1\}\forall r\in\mathscr{R}^T_t, v\in\mathscr{V}_t, trip\in Trip_{v,r,t} \tag{11}$$

where $\mathscr{R}^T_t$ denotes the set of transfer requests that must be dispatched in time stage t due to the waiting time and delay time constraints, and $x^l_{v,\,r,\,trip}$ is a binary parameter equal to 1 if and only if vehicle $v\in\mathscr{V}_l$ arrives at transfer station k in time stage t following $trip\in Trip_{v,\,r,\,l}$. Constraint set (8) ensures that each vehicle must take exactly one action. Constraint set (9) states that each waiting request is dispatched to at most one vehicle, while constraint set (10) ensures that each transfer request that must be served in this stage is dispatched to exactly one vehicle. Constraint set (11) guarantees that the number of transfer requests that will arrive at each transfer station in time stage t cannot exceed T^{max}. Constraint set (12) defines the binary variables $x^t_{v,\,r,\,trip}$.

2) Policy Based on Value Function Approximation

In this policy, we use the Bellman's equation to solve the problem. This policy takes future into account and usually makes decisions more efficiently in a long run. The optimal policy on each state S_t in time stage t tries to maximize the state value

$$\pi(S_t) = \underset{\mathscr{A}_t}{\operatorname{argmax}}(C_t(S_t,\mathscr{A}_t) + \gamma E\left\{V(S_{t+1})\,|S_t,\mathscr{A}_t\right\}), \tag{12}$$

where $V(\bullet)$ denotes the parameterized function, $\mathrm{E}\left\{V(S_{t+1})\,|S_t,\mathscr{A}_t\right\}$ is the expected value of state $S_{t+1} = T\left(S_t,\mathscr{A}_t,\mathscr{R}^{new}_{t+1}\right)$, and γ is a discount factor. To resolve Eq. (13), we attach the downstream cost to the post-decision state $S^{\mathscr{A}}_t$ instead of the next pre-decision state S_{t+1}, and replace the expected state value $\mathrm{E}\left\{V(S_{t+1})\,|S_t,\mathscr{A}_t\right\}$ with an approximation value $\overline{V}\left(S^{\mathscr{A}}_t\right) = E\left\{V(S_{t+1})\,|S^{\mathscr{A}}_t\right\}$. In other words, we find the optimal policy in stage t as follows:

$$\pi(S_t) = \underset{\mathscr{A}_t}{\operatorname{argmax}}\left(C_t(S_t,\mathscr{A}_t) + \gamma\overline{V}\left(S^{\mathscr{A}}_t\right)\right). \tag{13}$$

Considering that the value function of S_t equals the sum of the rewards of all the agents (vehicles) in this system, we write

$$\overline{V}\left(S^{\mathscr{A}}_t\right) = \sum_{v\in\mathscr{V}_t}\overline{V}_v\left(S^{\mathscr{A}}_{v,t}\right) = \sum_{v\in\mathscr{V}_t}\mathrm{E}\left\{\overline{V}_v\left(S_{v,t+1}\right)|S^{\mathscr{A}}_{v,t}\right\}, \tag{14}$$

where $S^{\mathscr{A}}_{v,t}$ is the post-decision state of vehicle v in time stage t.

Thus, we can find the optimal myopic policy in time stage t considering the future rewards by solving the following ILP model

$$max\sum_{r\in\mathscr{R}^T_t}\sum_{v\in\mathscr{V}_t}\sum_{trip\in Trip_{v,r,t}} C_{v,t}\left(S_{v,t},Assign_{v,t}\,(r,trip)\right)x^t_{v,r,trip} + \gamma\overline{V}\left(S^{\mathscr{A}}_t\right)$$

$$= max\sum_{r\in\mathscr{R}^T_t}\sum_{v\in\mathscr{V}_t}\sum_{trip\in Trip_{v,r,t}}\left(C_{v,t}\left(S_{v,t},Assign_{v,t}\,(r,trip)\right)x^t_{v,r,trip} + \gamma\overline{V}_v\left(T^{\mathscr{A}}\left(S_{v,t},Assign_{v,t}\,(r,trip)\right)\right)\right)x^t_{v,r,trip}$$

subject to constraint sets (8)–(12).

4.2. Policy evaluation using neural network

A look-up table is usually used to store the state values, which is an effective method to deal with a relatively small state space (Xu et al., 2018). As mentioned above, the state vector in the problem under study including the additional information on the customers assigned to each vehicle encounters the "curse of dimensionality", which renders the look-up table method non-viable. Neural network is wildly used to approximate the value function when encountering the curse of dimensionality. In this paper we exploit neural network to approximate the value function. Since all the vehicles are homogeneous, they can share a common value function, and we use $V_{NN}(S,\theta)$ to approximate the state value of an individual vehicle in the sharing system in state S, where θ is the parameter of the neural network. The input state not only include the basic information on the focus vehicle, but also the information on the other vehicles and the requests waiting to be served. In the following we show how to train the neural network to make the approximation state value closer to the true state value.

We show the detailed training algorithm in Algorithm 1, which is analogous to that developed in Liang et al. (2021). In this algorithm, we exploit temporal-difference (TD) prediction to estimate the state value $\overline{V}_v(S,a)$, and construct a target network to enhance the stability of training. Specifically, we create a new network $V_{NN}'(S,\theta')$ as a copy of the neural network $V_{NN}(S,\theta)$, where the former network is called the target network and the latter one is referred to as the online network. The two networks share the same structure,

and the difference lies in that the parameter θ updates every step while θ' updates every C steps during the training process. With the target network, the TD prediction value in time stage t is

$$\widehat{V}_{NN}(S,a) = \begin{cases} C(S,a), & \text{if } t = T \\ C(S,a) + \gamma V'_{NN}(T(S,a), \theta'), & \text{otherwise} \end{cases}. \tag{15}$$

This method can be called one-step TD or TD(0), and the estimate state value will be updated as soon as state S has transitioned to a new state S'.

Another policy to enhance the stability of training we adopt is the experience replay mechanism (Liang et al., 2021). With this technique, we store the transition pairs $\{S, a, C(S,a), T(S,a)\}$ into a replay memory $\mathscr{N}$ with capacity N. During the implementation of the training algorithm, we randomly select a subset of transition pairs $\mathscr{D}$ from M in each iteration, and define the loss function $\mathscr{L}(\theta)$ as the mean squared error (MSE) between the TD prediction value and the estimated state value $V_{NN}(S,\theta)$ as follows:

$$\mathscr{L}(\theta) = \frac{1}{|\mathscr{D}|} \sum_{\{S,a,C(S,a),T(S,a)\} \in \mathscr{D}} (V_{NN}(S,\theta) - \widehat{V}_{NN}(S,a))^2. \tag{16}$$

We use the stochastic gradient descent method to update the parameter θ in each training step as follows:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathscr{L}(\theta)$$

where α is the learning rate.

To make sure that the training algorithm has the ability to explore the state space and guarantee convergence of the algorithm, we apply the ε-greedy policy. Specifically, instead of always choosing the optimal action in each decision epoch, we randomly choose a feasible action with probability ε, and a feasible greedy action with probability $1 - \varepsilon$. The probability ε decreases from 1 to 0.1 in steps of $\Delta\epsilon$, and then keeps unchanged at 0.1.

Algorithm 1. Policy Evaluation With Neural Network

Algorithm 1 Policy Evaluation With Neural Network

Initialize $\varepsilon = 1$; replay memory $\mathcal{N}$ with capacity N; evaluate $V_{NN}(S,\theta)$ with an arbitrary θ; target network $V'_{NN}(S,\theta')$ with $\theta' = \theta$;
for *episode*=1 to k (the number of training episodes) **do**
  Initialize the ride system simulator
  for t=0 to T **do**
    get the state $S_t = (\mathcal{R}_t, \mathcal{V}_t)$ from the simulator
    for each vehicle $v \in \mathcal{V}_t$:
      determine the feasible action set $\mathcal{A}^f_{v,t}$ for vehicle v
      compute $C(S_{v,t}, a_{v,t})$, $v' = T^v(v, a_{v,t})$ for each action $a_{v,t}$ in $\mathcal{A}^f_{v,t}$
    randomly choose a value z from (0,1);
    if $z < \epsilon$:
      choose a random feasible action $\mathcal{A}_t$ with probability ϵ,
    else get the optimal action $\mathcal{A}_t$ by sovling the resulting ILP
    update the ride-sharing simulator based on $\mathcal{A}_t$;
    for each vehicle $v \in \mathcal{V}_t$:
      store the experience $[v, r(a_{v,t}), a_{v,t}, v' = T^v(v, a_{v,t})]$ to replay memory $\mathcal{N}$
    sample a random subset of experience $\mathcal{D}$ from $\mathcal{N}$
    calculate the loss function $\mathcal{L}(\theta)$ with subset $\mathcal{D}$
    update θ with the gradient descent method: $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$
    if $\epsilon > 0.1$:
      update $\epsilon \leftarrow \epsilon - \Delta\epsilon$

To limit the state space and reduce computation complexity for policy convergence, we show the detailed training framework in Fig. 2. The framework consists of the following three parts: environment unit, agent unit, and dispatching generation unit. The environment unit is the ride-sharing system simulator, which updates the states of the vehicles and the information on requests over

time. The agent unit is a learning module, whose main functions include updating the experience memory, training the estimated value function, and evaluating the estimated state value. The dispatching generation unit mainly generates the optimal actions for all the vehicles in the system with the estimated state value function in a specific environment. In each decision epoch, with the information on the waiting requests and new states of the vehicles, a feasible trip is generated and the state value is found by the state value function for each feasible action of each vehicle. With this information, an optimal dispatch action for all the vehicles in this decision epoch is obtained by solving the resulting ILP model. Then the system advances to the next time stage.

5. Numerical studies and result analysis

In this section we discuss the numerical studies based on a real-world dataset. We first provide the details and the dataset used, and then analyze the stability and performance of the algorithm with the dataset.

5.1. Dataset description and experience setup

We present the dataset used in this paper in Section 5.1.1 and discuss the parameter settings of the numerical studies in Section 5.1.2.

5.1.1. Dataset description

We use the centre of Chengdu (lat:103.98–104.2, lon:30.58–30.73), capital of the eastern province of Sichuan, China, as the study area. First, we collect the road network data from www.openstreetmap.org, and depict the final road network in Fig. 3. Second, we obtain the on-demand request dataset from DiDi Chuxing over the one-month period 1–30 November 2016. Each request contains the following information: request time, request location, and destination. We match the original location and destination to the nearest location in the road network with <200 m and discard the requests that cannot be matched. The dataset also contains the service time of each request, and we delete the requests with actual service times less than five minutes. In the end, we obtain 552,6510 requests in total, which accounts for >70% of the total requests.

5.1.2. Experience setup

We divide the study area into 82 hexagonal grids, each of which is comprised of several adjacent locations. Similar to Singh et al. (2022), we choose to evenly distribute the transfer stations in the study area, where every two transfer grids will be separated by a grid that guarantees that the transfer stations can be evenly distributed throughout the service area. Thus, we have 27 transfer stations in total. We show the divided hexagonal grids in Fig. 3, where the green grids present the chosen transfer stations. We set the maximum passenger capacity of each vehicle $\mathcal{N}^{max}$ to 2, the maximum transfer service capacity of each transfer station T^{max} to 20, the maximum waiting time Δ_w to 300 s, the maximum delay time Δ_d to 600 s, and the speed of each vehicle to 25 km/h. Considering the differences in traffic conditions between working days and rest days, we further restrict the dataset to weekdays, and set a weekday as an episode in the training algorithm. In this way, the requests dataset includes 22 weekdays, and $T = 1440$. From the request dataset, we randomly select about 50,000 requests at a time that compose a training episode.

We train our algorithm using requests data of 15 weekdays over the period 2–22 November 2016, and use the data on 1 November 2016 for validation. We set the reward R_r for each request r to 5, and other parameters in Eq. (1) as follows: $w_1 = 0.5$, $w_2 = 0.5$, and $w_3 = 0.1$, which can be changed according to the preference of the decision maker in real-life implementation. At the beginning of each training episode, we reset the ride-sharing system in a similar way, emptying all the requests in the environment and randomly assign a position for each vehicle as its initial position.

To limit the state space and reduce the computation complexity, we use one-hot representation of grids as each location input instead of directly using locations, and use low-dimensional embedding instead of one-hot representation for the current time stamp

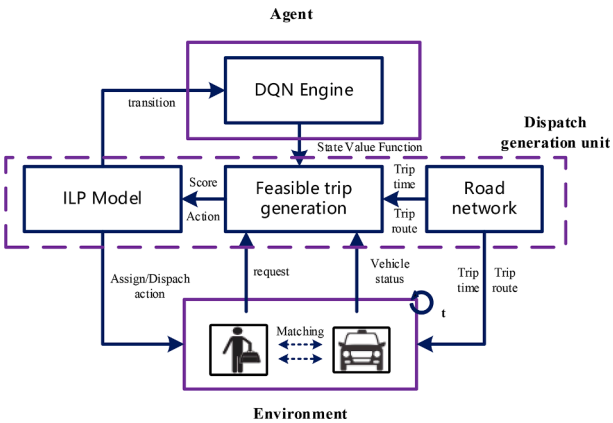

Fig. 2. Training framework.

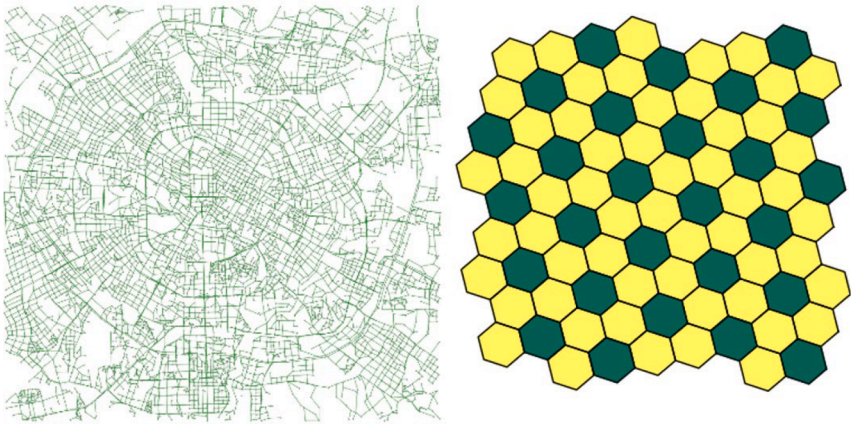

Fig. 3. The road network and the divided grids.

input since there are too many time slots in one episode.

We build the ride-sharing system simulator using Python 3.8 and solve the ILP model in Eq. (15) by Gurobi. The simulator updates at the end of each time stage, including the states of all the vehicles and requests. In each decision epoch, the new requests are collected and sent to the simulator, and the requests are removed if they are refused or completed by the vehicles.

5.2. Stability of training and performance of approximation value function

The effectiveness of the proposed RL method mainly depends on the approximation value function. We first analyze the convergence of the trained approximation value function by a neural network. Fig. 4 shows the accumulated reward of the vehicle fleet in each epoch during training. The result demonstrates that the cumulative reward converges in the eighth epoch. This result highlights the effectiveness of the trained approximation value function, which can generate stable state values very quickly.

Fig. 5 depicts the changes in the state-value function estimation at the end of all the training episodes with the variables of the global time and the vehicle number around normal grids. Fig. 5(a) gives the approximation state values in different times from time 0 to 1440 in a weekday. From Fig. 5(a), we observe that state value decreases gradually over time, since the state-value function is equal to the sum of the expected discounted rewards up to the end of the global time. We also see that the changes in the cumulative reward in the first 360 min (before 6:00 am) is not obvious since the request number becomes small after the midnight. Fig. 5(b) shows the changes in the cumulative reward with respect to the number of vehicles around a vehicle.

Fig. 6 shows the estimated state values of the vehicles in different grids at 6:00 pm. We see that the estimated state values in the central grids, which have higher demands, and that the estimated state value in each transfer station is higher than those of the grids around it. The reason for the latter observation lies in that for each transfer station, except for the new arrival requests, there are also transfer requests that will be transferred to this station, resulting in the requests in transfer stations being generally more than those in the grids around them.

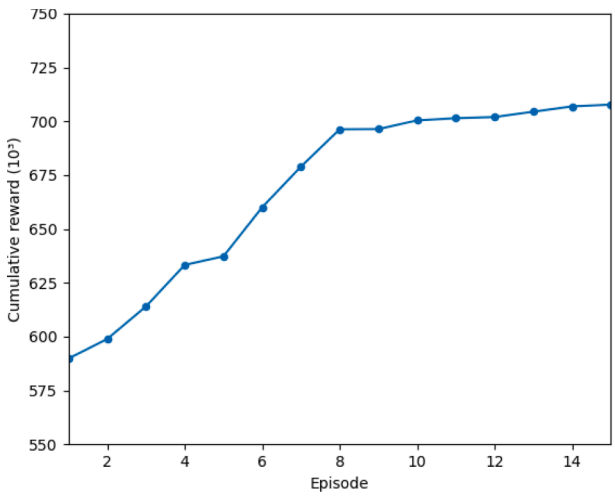

Fig. 4. Cumulative reward of vehicle fleet during training.

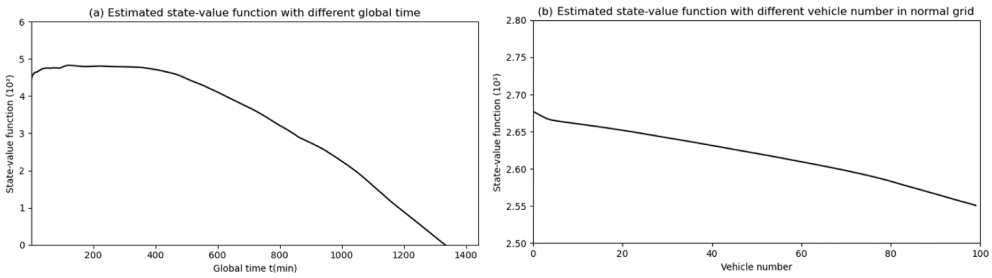

Fig. 5. Performance of the approximation value function.

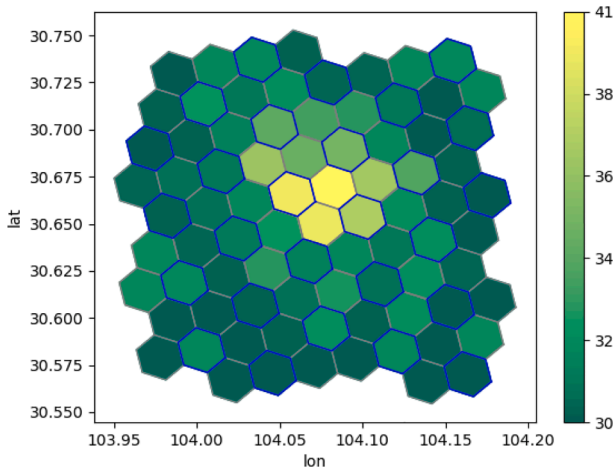

Fig. 6. The estimated state values in different grids in peak hours.

5.3. Performance comparisons

In this section, we assess the value of considering passenger transfer, and the effectiveness and efficiency of the developed method.

5.3.1. Value of considering passenger transfer

To assess the value of considering passenger transfer, we compare our ride-sharing model with passenger transfer (RST) with the traditional ride-sharing model (RS). To compare the differences in the model performance, we consider the following five metrics: (i) cumulative reward; (ii) number of served requests; (iii) matching rate, which is defined as the ratio between the number of served

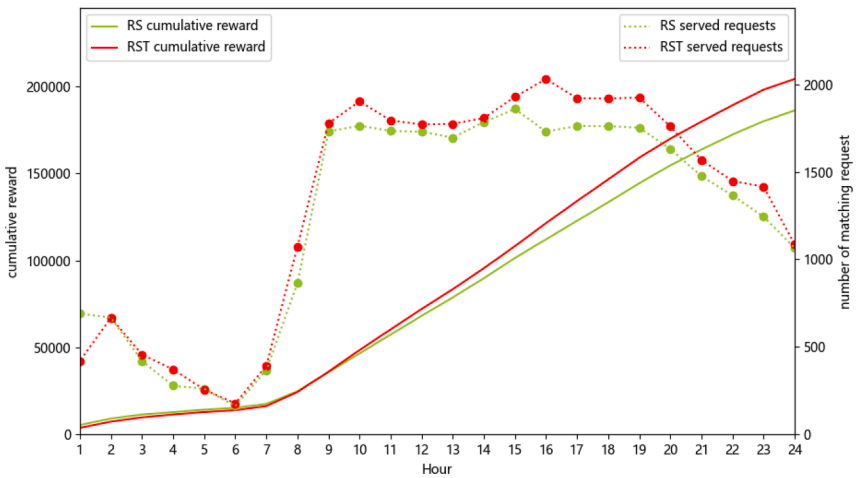

Fig. 7. Comparison results on cumulative reward and number of served requests between RS and RST.

requests and the number of the new requests; (iv) average delay time; and (v) utilization rate of seat.

Figs. 7 and 8 report the changes in the cumulative reward, number of served requests, matching rate, and average delay time with variations in time from 0:00 to 12:00 pm in every hour under each comparison model, respectively. The results clearly demonstrate that RST outperforms ST in terms of all performance measures in general. Indeed, RST yields the more cumulative reward after 8:00 am, and the more served requests after 2:00 am, the higher matching rate after 2:00 am, and the larger average delay time after 8:00 am. Only before 8:00 (resp., 2:00, 2:00) am is the cumulative reward (resp., number of served requests, matching rate) obtained from RST smaller than that from RS. In addition, only before 8:00 am is the average delay time obtained from RST-Learning significantly smaller that from RS. Fig. 7 also shows a common variation trend for the number of served requests: the peak values occur at 2:00 am, 10:00 am, and 4:00 pm or 5:00 pm, and the valley values occur at 6:00 am and 1:00 pm, where the maximum value and minimum served requests are achieved at 3:00 pm or 4:00 pm, and 6:00 am, respectively. The reason may be that the number of requests is the least between 2:00 am and 6:00 am, and becomes gradually more after 6:00 am. However, the traffic becomes congested between 6:00 am and 10:00 am, resulting in the number of served requests not being too large. On the other hand, we also observe a common variation trend for the cumulative reward: the rate of increase in the cumulative reward tapers off up to a certain threshold (3:00 am) and then increases sharply beyond a certain threshold (7:00 am). Fig. 8 illustrates that both the matching rate and average delay time remain relatively stable after 9:00 am. Before 7:00 am, both the matching rate and average delay time are larger those after 7:00 am, respectively. The reason behind this observation is that the number of requests is smaller and the traffic is less congested before 7:00 am.

Fig. 9 represents the utilization rate of seat of the two models, and shows the percentages of vehicles that are idle, serve one request, and simultaneously serve two requests at different times under each comparison model, respectively. The results demonstrate that RST achieves a higher percentage of vehicles that simultaneously serve two requests and the lowest percentage of vehicles that are idle, especially in the peak hours, such as 6:00 pm. This observation highlights that allowing passage transfer indeed leads to more requests that share vehicles, and a higher seat utilization. For example, during the peak times such as 6:00 pm, 40% of the vehicles simultaneously serve two requests and 80% of the vehicles serve at least one request.

Overall, the detailed analysis clearly demonstrate that the effectiveness of the strategy of ride-sharing with passenger transfer, which can significantly increase the cumulative reward, enhance seat utilization and matching rate, and reduce the average delay time in general, compared to the traditional ride-sharing model.

5.3.2. Comparison with model-free based DRL algorithm

We now compare our joint decision framework combining deep reinforcement learning with integer-linear programming (DRL-ILP) against the model-free based DRL algorithm developed in Singh et al. (2022), to which we refer as DRL-WM. DRL-WM does not take joint decisions of the whole vehicle fleet into account in the decision stages to avoid action space explosion, where a transfer station is selected in advance for each request and each agent in the environment takes the optimal decision to maximize self-interest without considering the other agents.

Fig. 10 reports the changes in the number of served requests and the average delay time of the served requests with variations in time from 0:00 to 12:00 pm in every hour under DRL-ILP and DRL-WM. From Fig. 10, as expected, we see that the average delay time of the served requests obtained from DRL-ILP is far smaller than that from DRL-WM. The cumulative number of served requests is almost the same before 9:00 am, but the cumulative number of served requests obtained from DRL-ILP is far more than that from DRL-WM after 9:00 am, both of which increase in an almost linear fashion. This observation demonstrates the superiority of DRL-ILP, which can dramatically enhance the service efficiency of the ride-sharing system and customer satisfaction by significantly increasing the number of served requests and reducing the delay time of customers, versus DRL-WM. Specifically, DRL-ILP increases the number of served requests by about 20.02% on average, and reduces the delay time by up to 68.21% on average, over DRL-WM.

Overall, the above results clearly attest to the effectiveness and efficiency of the developed method, which outperforms the model-free based DRL algorithm developed for solving a similar ride-sharing problem with passenger transfer.

6. Conclusions and outlook

We study a special ride-sharing model with transfer stations, which allows passenger transfer between vehicles at transfer stations such that the trip of a request can be divided into more than one segment. We model the problem as a Markov decision process, and develop an efficient DRL model combined with ILP to find the optimal dispatch in each decision epoch. The numerical results based on the real-world trip dataset in Chengdu demonstrate that ride-sharing with passenger transfer is more beneficial than traditional ride-sharing, which can efficiently increase the reward, enhance seat utilization and matching rates, and reduce passengers' delay times, and that the proposed method outperforms the model-free based DRL algorithm in Singh et al. (2022).

Future research may consider the following extensions to our study. First, researchers can extend our model to consider electric vehicles. Second, without limiting to transfers between sharing vehicles, our model is also applicable to share between sharing vehicles and public transport, which takes advantages of these two kinds of transport means. Finally, while our model is more beneficial than traditional ride-sharing, how to convince customers to accept the model is a more important problem. Thus, it is of interest to investigate the business model to promote ride-sharing with passenger transfer.

Author statement

All authors have made substantial contributions to the paper. The specific contributions of each author are as follows:

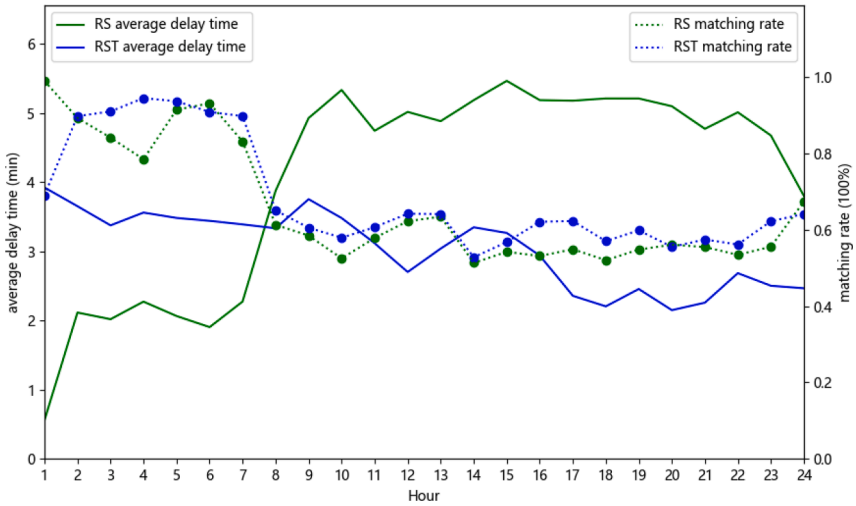

Fig. 8. Comparison results on matching rate and average delay time between RS and RST.

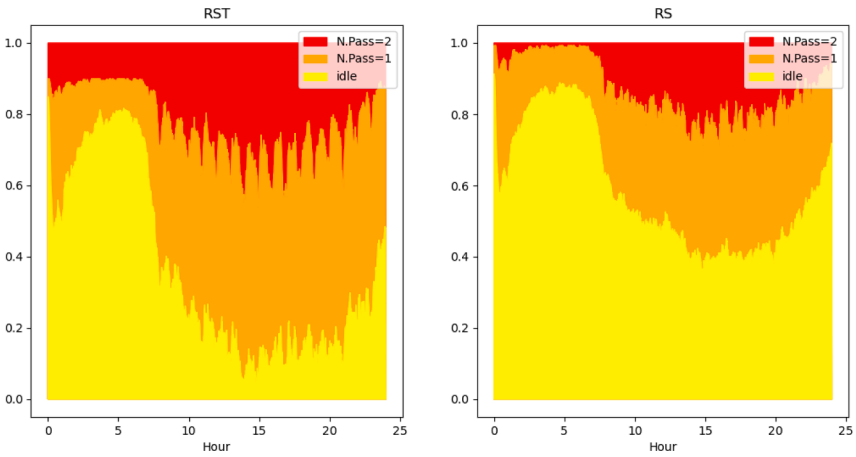

Fig. 9. Percentages of vehicles at different times in one day.

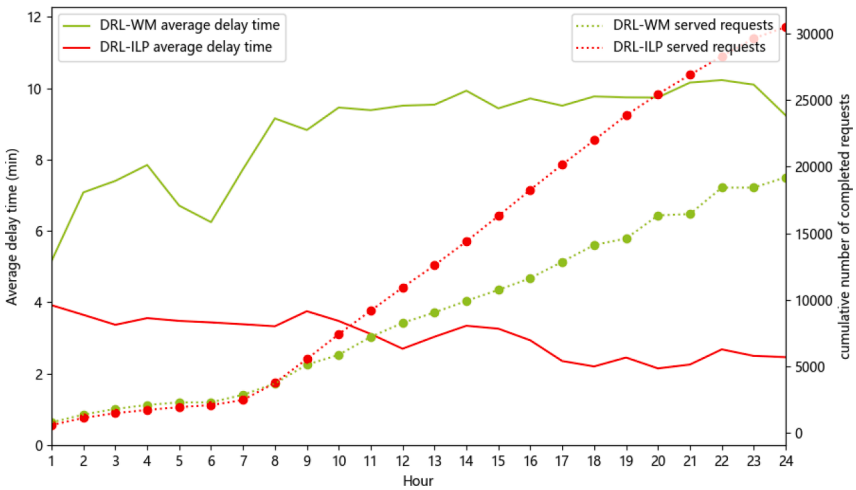

Fig. 10. Comparison results on number of served requests and average delay time between DRL-ILP and DRL-WM.

(1) Dujuan Wang provided the conceptualization, developed mathematical model and solution method, and drafted the original draft.
(2) Qi Wang performed model analysis, sample preparation and data analysis.
(3) Yunqiang Yin provided supervision and constructive comments, and provided the final approval of the version to be submitted and funding acquisition.
(4) T.C.E. Cheng reviewed and edited the paper critically for important intellectual content.

All authors read and contributed to the paper.

Acknowledgments

This study was supported in part by the National Key R&D Program of China (No. 2018AAA0101003); by the National Natural Science Foundation of China (Nos. 72171161, 71971041, and 71871148); by the Major Program of National Social Science Foundation of China (No. 20&ZD084); by the Key Research and Development Project of Sichuan Province (No., 2023YFS0397); by the Chengdu Philosophy and Social Science Planning Project (No., 2022C19); and by the Sichuan University to Building a World-class University (No. SKSYL2021-08).

References

Agatz, N., Erera, A., Savelsbergh, M., Wang, X., 2012. Optimization for dynamic ride-sharing: a review. Eur. J. Oper. Res. 223 (2), 295–303.
Al-Abbasi, A.O., Ghosh, A., Aggarwal, V., 2019. DeepPool: distributed model-free algorithm for ride-sharing using deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. 20 (12), 4714–4727.
Alisoltani, N., Leclercq, L., Zargayouna, M., 2021. Can dynamic ride-sharing reduce traffic congestion? Transp. Res. B Methodol. 145, 212–246.
Al-Kanj, L., Nascimento, J., Powell, W.B., 2020. Approximate dynamic programming for planning a ride-hailing system using autonomous fleets of electric vehicles. Eur. J. Oper. Res. 284 (3), 1088–1106.
Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E., Rus, D., 2017. On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. Proc. Natl. Acad. Sci. 114 (3), 462–467.
Asadi, A., Pinkley, S.N., 2021. A stochastic scheduling, allocation, and inventory replenishment problem for battery swap stations. Transp. Res. E Logist. Transp. Rev. 146, 102212.
Azcuy, I., Agatz, N., Giesen, R., 2021. Designing integrated urban delivery systems using public transport. Transp. Res. E Logist. Transp. Rev. 156, 102525.
Bertsimas, D., Jaillet, P., Martin, S., 2019. Online vehicle routing: the edge of optimization in large-scale applications. Oper. Res. 67 (1), 143–162.
Braverman, A., Dai, J.G., Liu, X., Ying, L., 2019. Empty-car routing in ridesharing systems. Oper. Res. 67 (5), 1437–1452.
Cao, Z., Guo, H., Song, W., Gao, K., Chen, Z., Zhang, L., Zhang, X., 2020. Using reinforcement learning to minimize the probability of delay occurrence in transportation. IEEE Trans. Veh. Technol. 69 (3), 2424–2436.
Chen, X., Zheng, H., Ke, J., Yang, H., 2020. Dynamic optimization strategies for on-demand ride services platform: surge pricing, commission rate, and incentives. Transp. Res. B Methodol. 138, 23–45.
Cortés, C.E., Matamala, M., Contardo, C., 2010. The pickup and delivery problem with transfers: formulation and a branch-and-cut solution method. Eur. J. Oper. Res. 200 (3), 711–724.
Feng, G.Y., Kong, G.W., Wang, Z.Z., 2021. We are on the way: analysis of on-demand ride-hailing systems. Manuf. Serv. Oper. Manag. 23 (5), 1237–1256.
Feng, S.Y., Duan, P.B., Ke, J.T., Yang, H., 2022. Coordinating ride-sourcing and public transport services with a reinforcement learning approach. Transp. Res. C Emerg. Technol. 138, 103611.
Fielbaum, A., 2021. Optimizing a vehicle's route in an on-demand ridesharing system in which users might walk. J. Intell. Transp. Syst. 26 (4), 432–447.
Fielbaum, A., Bai, X., Alonso-Mora, J., 2021. On-demand ridesharing with optimized pick-up and drop-off walking locations. Transp. Res. C Emerg. Technol. 126. Article 103061.
Haliem, M., Mani, G., Aggarwal, V., Bhargava, B., 2021. A distributed model-free ride-sharing approach for joint matching, pricing, and dispatching using deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. 22 (12), 7931–7942.
Hawkins, A., 2015. UberHop is Uber's latest idea for killing mass transit. [online]. Available: https://www.theverge.com/2015/12/8/9873544/uber-hop-commute-mass-transit-seattlechicago.
Holler, J., Vuorio, R., Qin, Z.W., Tang, X.C., Jiao, Y., Jin, T.C., Singh, S., Wang, C.X., Ye, J.P., 2019. Deep reinforcement learning for multi-driver vehicle dispatching and repositioning problem. In: 19th IEEE International Conference on Data Mining (ICDM), pp. 1090–1095.
Jiao, Y., Tang, X.C., Qin, Z.W., Li, S.J., Zhang, F., Zhu, H.T., Ye, J.P., 2021. Real-world ride-hailing vehicle repositioning using deep reinforcement learning. Transp. Res. C Emerg. Technol. 130, 103289.
Ke, J.T., Yang, H., Zheng, Z.F., 2020. On ride-pooling and traffic congestion. Transp. Res. B Methodol. 142, 213–231.
Ke, J.T., Zheng, Z.F., Yang, H., Ye, J.P., 2021. Data-driven analysis on matching probability, routing distance and detour distance in ride-pooling services. Transp. Res. C Emerg. Technol. 124, 102922.
Li, C., Parker, D., Hao, Q., 2021. Optimal online dispatch for high-capacity shared autonomous mobility-on-demand systems. In: IEEE International Conference on Robotics and Automation (ICRA) (ICRA 2021), pp. 779–785.
Liang, Y.C., Ding, Z.H., Ding, T., Lee, W.-J., 2021. Mobility-aware charging scheduling for shared on-demand electric vehicle fleet using deep reinforcement learning. IEEE Trans. Smart Grid 12 (2), 1380–1393.
Lin, K.X., Zhao, R.Y., Xu, Z., Zhou, J.Y., 2018. Efficient large-scale fleet management via multi-agent deep reinforcement learning. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1774–1783.
Lokhandwala, M., Cai, H., 2018. Dynamic ride sharing using traditional taxis and shared autonomous taxis: a case study of NYC. Transp. Res. C Emerg. Technol. 97, 45–60.
Manchella, K., Umrawal, A.K., Aggarwal, V., 2021. FlexPool: a distributed model-free deep reinforcement learning algorithm for joint passengers and goods transportation. IEEE Trans. Intell. Transp. Syst. 22 (4), 2035–2047.
Masoud, N., Jayakrishnan, R., 2017. A decomposition algorithm to solve the multi-hop peer-to-peer ride-matching problem. Transp. Res. B Methodol. 99, 1–29.
Ministry of Transport of the People's Republic of China, 2021. https://www.mot.gov.cn/jiaotongyaowen. Accessed on 19 May 2021.
Narayanan, S., Chaniotakis, E., Antoniou, C., 2020. Shared autonomous vehicle services: a comprehensive review. Transp. Res. C Emerg. Technol. 111, 255–293.
Santi, P., Resta, G., Szell, M., Sobolevsky, S., Strogatz, S.H., Ratti, C., 2014. Quantifying the benefits of vehicle pooling with shareability networks. Proc. Natl. Acad. Sci. 111 (37), 13290–13294.
Shah, S., Lowalekar, M., Varakantham, P., 2020. Neural approximate dynamic programming for on-demand ride-pooling. In: 34th AAAI Conference on Artificial Intelligence, pp. 507–515.
Shi, J., Gao, Y.Q., Wang, W., Yu, N.P., Ioannou, P.A., 2020. Operating electric vehicle fleet for ride-hailing services with reinforcement learning. IEEE Trans. Intell. Transp. Syst. 21 (11), 4822–4834.

Simonetto, A., Monteil, J., Gambella, C., 2019. Real-time city-scale ridesharing via linear assignment problems. Transp. Res. C Emerg. Technol. 101, 208–232.
Singh, A., Al-Abbasi, A.O., Aggarwal, V., 2022. A distributed model-free algorithm for multi-hop ride-sharing using deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. 23 (7), 8595–8605.
Tang, X.C., Qin, Z.W., Zhang, F., Wang, Z.D., Xu, Z., Ma, Y.T., Zhu, H.T., Ye, J.P., 2019. A deep value-network based approach for multi-driver order dispatching. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1780–1790.
Teubner, T., Flath, C.M., 2015. The economics of multi-hop ride sharing creating new mobility networks through IS. Bus. Inf. Syst. Eng. 57 (5), 311–324.
Uber, 2022. Shared-rides. https://www.uber.com/us/en/marketplace/matching/shared-rides. Accessed on 19 May 2021.
Wang, S.C., Noland, R.B., 2021. Variation in ride-hailing trips in Chengdu, China. Transp. Res. Part D: Transp. Environ. 90, 102596.
Wang, H., Yang, H., 2019. Ridesourcing systems: a framework and review. Transp. Res. B Methodol. 129, 122–155.
Xu, Z., Li, Z.X., Guan, Q.W., Zhang, D.S., Li, Q., Nan, J.X., Liu, C.Y., Ye, J., 2018. Large-scale order dispatch in on-demand ride-hailing platforms. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 905–913.
Yao, D.L., He, S.M., Wang, Z., 2021. A new ride-sharing model incorporating the passengers' efforts. Nav. Res. Logist. 68 (4), 397–411.
Yu, X., Shen, S.Q., 2020. An integrated decomposition and approximate dynamic programming approach for on-demand ride pooling. IEEE Trans. Intell. Transp. Syst. 21 (9), 3811–3820.